

App Dev Project Report

1. Student Details

Name: Sumit Goyal

Roll Number: 21f3002375

Email: 21f3002375@ds.study.iitm.ac.in

About Me: I am a working professional currently based in Hyderabad as an Associate Software Engineer, also completed my BTech from Chitkara University last year while doing this course as a dual degree.

2. Project Details

Project Title: Quantified Self App

Problem Statement:

Adventure organizations often rely on spreadsheets, phone calls, and manual coordination to manage trekking activities. These methods make it difficult to handle trek registrations, approve staff, assign trekking guides, prevent overbooking, and maintain trekking history.

The objective of this project is to develop a **web-based Trekking Management Application** that enables Admin, Trek Staff, and Users (Trekkingers) to efficiently manage trekking activities through a role-based system.

Approach:

The application was developed using a modular architecture by separating the project into Models, Routes, Templates, and Static resources.

The development process followed these steps:

1. Database models were created using SQLAlchemy.
2. User authentication was implemented using Flask-Login.
3. Separate dashboards were created for Admin, Staff, and Users.
4. CRUD operations were implemented for trek management.
5. Booking functionality was developed with slot validation to prevent overbooking.
6. Role-based authorization was implemented to ensure that only authorized users could perform specific actions.

3. AI/LLM Declaration

Artificial Intelligence (**ChatGPT (GPT-5)**) was used as a learning and guidance tool during the development of this project. AI assistance was primarily used to understand Flask concepts, debug errors, explain SQLAlchemy relationships, and improve the project structure. The project implementation, integration, testing, debugging, and final submission were completed by me. I understand the submitted code and can explain its functionality during the viva. The extent of AI/LLM usage is around **25–30%**.

4. Technologies and Frameworks Used

Technology / Library	Purpose
Flask	Core backend web framework
SQLAlchemy	Object Relational Mapper for SQLite database
Jinja2	HTML Templating
Bootstrap 5	Frontend styling and responsive design
HTML5	Structure of Web Pages
Flask-Login	User authentication and session management
Werkzeug Security	Password Hashing
SQLite	Lightweight local database for storing user data

5. Database Schema / ER Diagram

Users

- id (PK), name, email, password, phone, role, status

Treks

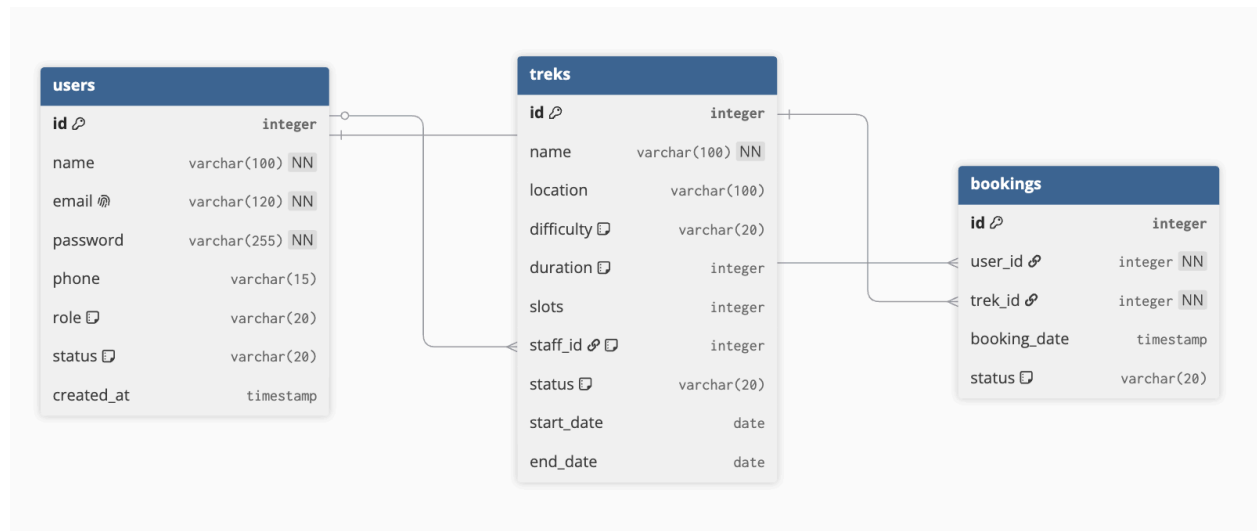
- id (PK), name, location, difficulty, duration, slots, staff_id (FK), status, start_date, end_date

Bookings

- id (PK), user_id (FK), trek_id (FK), booking_date, status

Relationships

- One User can have multiple Bookings (One-to-Many).
- One Trek can have multiple Bookings (One-to-Many).
- One Staff member can be assigned to multiple Treks (One-to-Many).
- Each Booking belongs to exactly one User and one Trek (Many-to-One).



6. API Resource Endpoints

HTTP Method	Route Endpoint	Description
GET / POST	/login	User Login
GET / POST	/register	User and Staff Registration
GET	/logout	Logout from the application
GET	/admin	Admin Dashboard
GET / POST	/admin/add_trek	Add a New Trek
GET / POST	/admin/edit/<id>	Edit Trek Details
GET	/admin/delete/<id>	Delete a Trek
GET	/admin/bookings	View All Bookings
GET	/admin/users	View All Registered Users

GET	/admin/staff	View and Manage Staff
GET	/admin/search	Search Treks
GET	/staff	Staff Dashboard
GET / POST	/staff/update/<id>	Update Trek Details
GET	/staff/participants/<id>	View Trek Participants
GET	/user	User Dashboard
GET	/user/book/<id>	Book a Trek
GET	/user/bookings	View Booking History
GET / POST	/user/profile	Edit User Profile

7. Architecture and Features

Architecture Overview

- **app.py** – Main Flask application entry point responsible for application initialization and route registration.
- **config.py** – Stores application configuration, including database and secret key settings.
- **models/** – Contains SQLAlchemy database models (**User**, **Trek**, and **Booking**).
- **routes/** – Flask Blueprints for Authentication, Admin, Staff, and User functionalities.
- **templates/** – Jinja2 HTML templates for dynamic web pages.
- **static/** – Stores CSS files and static assets used for styling the application.
- **SQLite Database** – Stores user, trek, and booking information using SQLAlchemy ORM.

Implemented Features

- User and Trek Staff registration with secure login.
- Role-based authentication for Admin, Staff, and User.
- Admin dashboard with trekking statistics.
- Trek management (Add, Edit, Delete, and View Treks).
- Staff approval and assignment to treks.
- Trek booking with overbooking prevention.
- Booking history for users.
- Staff dashboard for managing assigned treks.
- Participant management for assigned treks.
- Search functionality for treks.
- User profile update.
- Secure password hashing and session management using Flask-Login.

8. Conclusion

The Trekking Management Application successfully digitizes the trekking management process by providing separate interfaces for Admin, Trek Staff, and Users. The application simplifies trek management, participant booking, staff assignment, and booking history while preventing overbooking through backend validation. The project demonstrates the practical implementation of Flask, SQLAlchemy, SQLite, authentication, role-based access control, and relational database management.

9. Video Presentation

Click the link below to view the working demo for this trekking application.

https://drive.google.com/file/d/14dunAZpEI4to2WigHD46rezzQ7uPSLia/view?usp=drive_link